Avaliação A2 - Paradigmas de Linguagens de Programação

Grupo 08

■ Integrantes

- Gustavo Oliveira Castro (34032797)
- Kaio do Nascimento Mendes

Definição do Tema

Paradigma: Funcional

Linguagem: Haskell

Código do Tema: F2

Descrição do Desafio: Pipeline de dados com map/filter/fold para processamento de notas, frequências e estatísticas acadêmicas de estudantes.

Execução do Código

Ambiente Online Utilizado: tio.run

Permalink: https://tio.run/##rVbvTtxGEP9@TzGymsgucNwBl1RIF8kNIaG6g/Yg@QKomtgLt629e1nbELX0KfoI/VC1r8GD0dldn732GUJQLYF969/M/OavZ47ZryxJ7u5izBHCpBASxuX99x7QJWTK6La7C8e54uJyvTzNMRzq0z1ZfEyYc7rVPr3AJMdMnx6IvPcHxEzxK9IE/vFcXge9nrE9Y1lBuFguSbQOLBt1aOlAN6V9LjCxrxoMFPtUMBFxXHmV8bzACGVDYRfJjQ2Y8IyIxgxiYpXpB/aZpYtE9lBTzN7YX1rPqSF93noxhlNj1HrkhSmKGOEDZ1yhBzv9Ldjufwcv1gni4r5XjG6vUZFV6cGwP4Kt/gCGGujiXmPKE8qiyAvBMg9GBNrR4BZuDwVnCUyZiDVsm2BbGtbG7TMlDMMJTw0/skn8hm3cW/yotMJjFLkkhVrZNv0NR@sN3DuWSJ5RqmU0R01vZHTutP09uJQKjnlyZeOifdVxGQ5adn8oEo4CYSoVWke2SOEQ2rhJEVH9nTD@2SocmriQ0yONO@8tlIxYlqGyeEqffdh41SrCNhK7S9U3PWNeBpBTWSaH9A8WUkVM5HjJ0v26IKmk8iIjutdzppihXYuMl41mLa0tO8z8NFgqyzDLipSLWEKRWllYYKxu/5K6RF8MYE7xyTaxSCgKC1QICJEkItSZvzEFGZHiJB3JFHRZm@K@kEqwiI6z3aWd4WjZyizVaq1CxRaKaRVI3UCNnNIjo9A@08ZLvP/S/rTI278x6NeOhkbNmDT2B@a0O05jyj8hYAN8/0LJlGYJu1Tkq18aKeO96WgN4FsrFRjFVcB@KTLiSIRSVGQFFsYKgs8WGU@kCGyY2BXPURFrSeMiRdPxqBSLJTWF9pMCLMtpYoeJySXcOBl8RY4R6@fPwe/2aw2I32AwDDT0pW7ZMU0GsnlFefB6UF83IHOqkWueMXjCRWpnrNLbu@BJrqiQyxMzn0@bhXyuO6B9tiqolgCdRf2Waso/U1q4mq4Kxq5bgSNUUanYfTUXR/LxZOpgNNlEmERUPGrKYo46h/fSKTPfIeCyMClMWA6ZTMvXxEwmMZXuGUYRGHL6Yc35hqlA14VLTKvhwtGyCc0@SJi4zOeOSNByxqm6r/DIkbrXLY253yvn8/ugW0bLY7yi4ZTSqFETLuZYDe3WFCbz5de8A04lYMeBp0cC/V9bK3cLpR89ODXda3Kxa19ntAfo0e5kiE5LJTdgwnT7r/ayIdD0RslCxPqwjoiZUYEeXHQzxp@dU3@mSFHRO9MR@IH9NaZxYydkkZNrEwHe@CmX11IyezMJT27/nB0cef@H@jPh1eNWV0ijJ1NcQNdndLkkNUWxamzbzfcMHsfUqkDndGiasZ8MQ4PkVkquwn3K49AxUAIrki5uxlaBNbsKmdZDYwwPTpKmSNlxX@7U2tE6rXvh3tEx7IVw8n42DVtJ/wa8E7NE0NeuDEll9U6Lm5nMTm07UVtR8VOBeqvA2Hxn66DpNdtR4Qb0CzqcgK7omLF7lUxv/6HQaAU2oLTOnhQ0D9pMuhvVSY/bpg9YceYAvGVa12PNmJSuDgOvK4NnIpy8P6Qkhj/Ojj7odFoYvwBRJInTN9UGQbuDcDTQiX/IxJx2RrtcLkWCeulgSaZ3pMX0Z

9pelpJ9WJ2nQasHHmI8e9NN2WndR3OuZJ5GutWPnayfOGHv7v4D

Instruções de Teste (Entrada e Saída)

Entrada de Exemplo (Embutida no Código):

```
alunosExemplo = [
    Aluno "Amanda Vieira" 4.2 3.8 6,
    Aluno "Breno Cardoso" 1.5 2.0 16,
    Aluno "Camila Antunes" 5.0 4.5 2,
    Aluno "Daniel Mendes" 3.0 2.5 22,
    Aluno "Fernanda Lima" 4.0 1.8 12,
    Aluno "Gabriel Santos" 2.5 3.5 15,
    Aluno "Heloisa Rocha" 5.5 4.0 4,
    Aluno "Igor Silveira" 2.0 3.8 10,
    Aluno "Juliana Moraes" 3.2 4.1 0,
    Aluno "Lucas Teixeira" 1.0 4.0 25
 ]
```

Saída Esperada:

```
======================================================
RELATÓRIO
======================================================

DADOS DA TURMA
Total de Alunos Analisados : 10
Quantidade de Aprovados    : 6
Quantidade de Reprovados   : 4
Média de Notas da Turma    : 6.2
Média de Frequência Geral  : 81.3%

ALUNOS APROVADOS
   - Amanda Vieira [Nota Final: 8.0 | Frequência: 90.0%]
   - Camila Antunes [Nota Final: 9.5 | Frequência: 96.7%]
   - Gabriel Santos [Nota Final: 6.0 | Frequência: 75.0%]
   - Heloisa Rocha [Nota Final: 9.5 | Frequência: 93.3%]
   - Igor Silveira [Nota Final: 5.8 | Frequência: 83.3%]
   - Juliana Moraes [Nota Final: 7.3 | Frequência: 100.0%]

ALUNOS REPROVADOS
   - Breno Cardoso [Nota Final: 3.5 | Frequência: 73.3%]
   - Daniel Mendes [Nota Final: 5.5 | Frequência: 63.3%]
   - Fernanda Lima [Nota Final: 5.8 | Frequência: 80.0%]
   - Lucas Teixeira [Nota Final: 5.0 | Frequência: 58.3%]

======================================================
```

Relatório Técnico

Descrição da Solução: O sistema realiza a leitura de uma base de dados simulada contendo registros de estudantes, calcula suas respectivas notas finais e percentuais de frequência, determina a situação acadêmica de cada indivíduo (Aprovado ou Reprovado) e gera estatísticas agregadas da turma com base nos critérios estabelecidos pelo docente. O código faz uso de Tipos de Dados Algébricos (ADT) através da sintaxe de Record Syntax para definir as entidades de entrada (Aluno) e as estruturas de saída formatada (ResultadoAluno). O processamento de dados funciona como um pipeline declarativo contínuo, utilizando funções de alta ordem (map, filter, foldl) em substituição a variáveis mutáveis e laços de repetição imperativos.

Análise Comparativa

1. Sintaxe

O Haskell utiliza uma abordagem matemática limpa e concisa. A definição de estruturas de dados baseia-se em tipos algébricos puros, e as transformações ocorrem por meio de expressões encadeadas e funções anônimas (lambdas). Não existem as palavras-chave class, public ou a necessidade de criar métodos assessores (getters e setters). Em contrapartida, Java exige uma sintaxe mais prolixa baseada em classes, métodos e declarações explícitas de visibilidade. Em C, a sintaxe estruturada exigiria o uso manual de struct, laços de repetição (for/while) e manipulação de índices de vetores, resultando em um código puramente algorítmico e distante da declaração matemática de alto nível do Haskell.

2. Semântica

- **Tipos e Escopo:** Haskell possui um sistema de tipos estático forte com inferência avançada e garante a imutabilidade por padrão. O escopo de variáveis é puramente local às expressões (uso de blocos where e let/in). Java trabalha com tipagem estática e mutabilidade em escopos de bloco. O C possui tipagem estática fraca (permitindo coerções implícitas perigosas) e escopos baseados em variáveis globais/locais altamente mutáveis.
- **Forma de Avaliação:** Haskell opera sob o princípio da avaliação preguiçosa (lazy evaluation), calculando expressões apenas no momento exato em que seus resultados tornam-se estritamente necessários para a saída do programa. Java e C operam sob avaliação estrita (eager evaluation), resolvendo cada instrução e cálculo de forma sequencial imediata.
- **Estilo de Computação:** O pipeline em Haskell baseia-se na aplicação de funções puras sem efeitos colaterais. Em Java, o processamento ocorre via comandos de atribuição que modificam o estado interno dos objetos. Em C, a computação é totalmente imperativa e baseada em efeitos colaterais diretos na memória por meio de instruções sequenciais que alteram os valores das variáveis a cada iteração.

3. Gerenciamento de Memória

Em Haskell, o gerenciamento é feito por um Garbage Collector (GC) otimizado para lidar com alocações rápidas e de curtíssimo ciclo de vida, características comuns à criação constante de dados imutáveis intermediários em pipelines funcionais. Java também utiliza coleta de lixo automática (GC) para instâncias de objetos complexos e referências mutáveis que residem no espaço Heap, gerando dinâmicas distintas de varredura. Já a linguagem C não possui nenhum mecanismo de automação ou coleta de lixo. O gerenciamento de memória em C é inteiramente de responsabilidade do programador, exigindo alocação dinâmica explícita (malloc/calloc) e liberação manual obrigatória (free), o que expõe o código a riscos de vazamentos de memória (memory leaks) ou falhas de segmentação (segmentation faults) — problemas inexistentes no ambiente seguro do Haskell.

4. Trade-offs

• **Vantagens do Haskell:** O código expressa com precisão o que está sendo feito em alto nível (filtrar, mapear, reduzir) em vez de detalhar como fazer. A imutabilidade nativa elimina efeitos colaterais indesejados e garante previsibilidade matemática ao algoritmo. Em relação ao C, oferece uma segurança de memória incomparável e a eliminação de bugs de ponteiros.

• **Desvantagens do Haskell:** Apresenta uma curva de aprendizado inicial mais íngreme para desenvolvedores habituados ao fluxo de execução imperativo linear. Operações de Entrada e Saída (I/O) precisam ser isoladas no monad IO. Em comparação com o C, o Haskell gera um overhead devido ao seu tempo de execução (runtime) e pelas pausas de varredura do Garbage Collector, enquanto o C entrega velocidade de execução bruta superior e controle absoluto sobre o hardware.

Resumo da Análise Comparativa

• **Sintaxe:** Haskell elimina classes, métodos e loops imperativos em favor de tipos algébricos, expressões encadeadas e funções lambdas matemáticas.

• **Semântica:** Garante imutabilidade por padrão, tipagem forte com inferência, escopo puramente local e avaliação preguiçosa (lazy evaluation).

• **Gerenciamento de Memória:** Coleta de lixo (Garbage Collection) automática e otimizada para dados imutáveis de ciclo curto, eliminando a liberação manual necessária no C.

• **Trade-offs:** Ganha-se em expressividade de alto nível, previsibilidade e segurança de memória; perde-se no controle de baixo nível de hardware e no desempenho bruto imediato se comparado ao C.

Log de Uso de Inteligência Artificial (IA)

• **O que foi pedido à IA:** Foi feito um único comando inicial solicitando um algoritmo simples em Haskell que utilizasse uma pipeline de dados básica (map, filter, fold) para registrar notas de alunos.

• **Qual IA foi utilizada:** Copilot

• **O que foi aproveitado:** A ferramenta fornecen apenas um esqueleto conceitual e didático da aplicação de funções de alta ordem (um filter por nota, um map aplicando bônus e um foldl somando valores de tuplas simples). A ideia de encadeamento serviu apenas como ponto de partida conceitual para a arquitetura do pipeline.

• **O que foi reescrito/entendido pelo grupo:** Toda a lógica real de negócio e complexidade do sistema foi projetada e implementada de forma autoral pelo grupo: Modelagem Corporativa de Dados, Adaptação às Regras Institucionais e Tratamento de Ponto Flutuante com epsilon.